



Herramientas personalizadas

Cree herramientas que LLM pueda llamar en opencode.

Las herramientas personalizadas son funciones que usted crea y que el LLM puede llamar durante las conversaciones. Trabajan junto con las herramientas integradas de opencode como ``read``, ``write`` y ``bash``.

Crear una herramienta

Las herramientas se definen como archivos TypeScript o JavaScript. Sin embargo, la definición de la herramienta puede invocar secuencias de comandos escritas en cualquier idioma: TypeScript o JavaScript solo se utilizan para la definición de la herramienta en sí.

Ubicación

Se pueden definir:

Localmente colocándolos en el directorio ``.opencode/tools/`` de tu proyecto.

O de forma global, colocándolos en ``${~}/.config/opencode/tools/``.

Estructura

La forma más sencilla de crear herramientas es utilizar el asistente ``tool()`` que proporciona seguridad de tipos y validación.

```
.opencode/tools/database.ts
```

```
import { tool } from "@opencode-ai/plugin"

export default tool({
  description: "Query the project database",
  args: {
    query: tool.schema.string().describe("SQL query to execute"),
  },
  async execute(args) {
    // Your database logic here
    return `Executed query: ${args.query}`
  },
})
```

El nombre de archivo se convierte en el nombre de la herramienta. Lo anterior crea una herramienta `database`.

Múltiples herramientas por archivo

También puede exportar varias herramientas desde un solo archivo. Cada exportación se convierte en una herramienta independiente con el nombre `<filename>_<exportname>`:

`.opencode/tools/math.ts`

```
import { tool } from "@opencode-ai/plugin"

export const add = tool({
  description: "Add two numbers",
  args: {
    a: tool.schema.number().describe("First number"),
    b: tool.schema.number().describe("Second number"),
  },
  async execute(args) {
    return args.a + args.b
  },
})

export const multiply = tool({
  description: "Multiply two numbers",
  args: {
    a: tool.schema.number().describe("First number"),
    b: tool.schema.number().describe("Second number"),
  },
  async execute(args) {
    return args.a * args.b
  },
})
```

Esto crea dos herramientas: ``math_add`` y ``math_multiply``.

Colisiones de nombres con herramientas integradas

Las herramientas personalizadas están codificadas por el nombre de la herramienta. Si una herramienta personalizada utiliza el mismo nombre que una herramienta integrada, la herramienta personalizada tiene prioridad.

Por ejemplo, este archivo reemplaza la herramienta ``bash`` integrada:

```
.opencode/tools/bash.ts
```

```
import { tool } from "@opencode-ai/plugin"

export default tool({
  description: "Restricted bash wrapper",
  args: {
    command: tool.schema.string(),
  },
  async execute(args) {
    return `blocked: ${args.command}`
  },
})
```

ⓘ NOTA

Prefiera nombres únicos a menos que intencionalmente desee reemplazar una herramienta integrada. Si desea deshabilitar una herramienta integrada pero no anularla, use permisos.

Argumentos

Puedes usar `tool.schema`, que es simplemente Zod, para definir tipos de argumentos.

```
args: {
  query: tool.schema.string().describe("SQL query to execute")
}
```

También puedes importar Zod directamente y devolver un objeto simple:

```
import { z } from "zod"

export default {
  description: "Tool description",
  args: {
    param: z.string().describe("Parameter description"),
  },
  async execute(args, context) {
    // Tool implementation
    return "result"
  },
}
```

Contexto

Las herramientas reciben contexto sobre la sesión actual:

.opencode/tools/project.ts

```
import { tool } from "@opencode-ai/plugin"

export default tool({
  description: "Get project information",
  args: {},
  async execute(args, context) {
    // Access context information
    const { agent, sessionId, messageId, directory, worktree } = context
    return `Agent: ${agent}, Session: ${sessionId}, Message: ${messageID},
Directory: ${directory}, Worktree: ${worktree}`
  },
})
```

Utilice `context.directory` para el directorio de trabajo de la sesión. Utilice `context.worktree` para la raíz del árbol de trabajo de git.

Ejemplos

Escribir una herramienta en Python

Puede escribir sus herramientas en cualquier idioma que desee. Aquí hay un ejemplo que suma dos números usando Python.

Primero, cree la herramienta como un script de Python:

.opencode/tools/add.py

```
import sys

a = int(sys.argv[1])
b = int(sys.argv[2])
print(a + b)
```

Luego cree la definición de herramienta que la invoca:

.opencode/tools/python-add.ts

```
import { tool } from "@opencode-ai/plugin"
import path from "path"

export default tool({
  description: "Add two numbers using Python",
  args: {
    a: tool.schema.number().describe("First number"),
    b: tool.schema.number().describe("Second number"),
  },
  async execute(args, context) {
    const script = path.join(context.worktree, ".opencode/tools/add.py")
    const result = await Bun.$`python3 ${script} ${args.a} ${args.b}`.text()
    return result.trim()
  },
})
```

Aquí estamos usando la utilidad `Bun.$` para ejecutar el script Python.

 [Edita esta página](#)

© Anomaly

 [Found a bug? Open an issue](#)

Última actualización: 21 may 2026

 [Join our Discord community](#)

 [Español](#) 